A.4.3. constexpr function requirements

In order to declare a function as `constexpr` it must meet a few requirements; if it doesn't meet these requirements, declaring it `constexpr` is a compilation error. The requirements for a `constexpr` function are as follows:

- All parameters must be of a literal type.
- The return type must be a literal type.
- The function body must consist of a single `return` statement.
- The expression in the `return` statement must qualify as a constant expression.
- Any constructor or conversion operator used to construct the return value from the expression must be `constexpr`.

This is straightforward; you must be able to inline the function into a constant expression and it will still be a constant expression, and you must not modify anything. `constexpr` functions are *pure functions* with no side effects.

For `constexpr` class member functions there are additional requirements:

- `constexpr` member functions can't be virtual.
- The class for which the function is a member must be a literal type.

The rules are different for `constexpr` constructors:

- The constructor body must be empty.
- Every base class must be initialized.
- Every non-`static` data member must be initialized.
- Any expressions used in the member initialization list must qualify as constant expressions.
- The constructors chosen for the initialization of the data members and base classes must be `constexpr` constructors.
- Any constructor or conversion operator used to construct the data members and base classes from their corresponding initialization expression must be `constexpr`.